

Self-Supervised Methods for Computation of Optimal Regularisation Parameters in Variational Image Denoising

Mohammad Rayyan Hameed

Student Number: 231172804

Supervisor: Dr Kostas Papafitsoros

April 30, 2026



Abstract

This project focuses on Total Variation (TV) denoising and, in particular, the role of the regularisation parameter α . The choice of this parameter is crucial, as it determines the balance between noise removal and preservation of important image features. Many existing and widely studied approaches determine the regularisation parameter using supervised learning techniques. Although these methods can produce strong results, they rely heavily on the availability of ground truth data. This project aims to address this limitation by exploring self-supervised methods for selecting regularisation parameters. Unlike supervised methods, self-supervised approaches estimate suitable parameters directly from noisy observations, without requiring clean reference images. We begin by investigating and implementing several self-supervised loss functions and then proceed to evaluate them using standard image analysis metrics such as PSNR and SSIM. In addition we also explore the case of the spatially varying regularisation parameter to study how self-supervised methods can impact its effectiveness in image denoising.

Contents

1	Introduction and Background	4
1.1	Inverse Image Problems and Image Denoising	5
1.2	The Variational Approach	6
1.3	Total Variation Denoising	7
1.4	Review of Existing Literature	10
2	The Supervised Approach	10
2.1	Parameter Selection via Supervised Losses	10
3	Self-Supervised Approaches	11
3.1	Overall Strategy	11
3.2	The Foundation: Noise2Noise Loss	12
3.3	Recorrputed2Recorrputed Loss	13
3.4	Neighbour2Neighbour Loss	15
3.5	SURE Loss	18
4	The Case of The Spatially Varying Regularisation Parameter	21
4.1	The Main Idea	21
4.2	Practical Implementation	21
5	Implementation	22
5.1	Data	22
5.2	Chambolle-Pock Algorithm	23
5.3	Algorithm Implementation	23
5.4	Experimental Setup	25
6	Results	26
6.1	Global Parameter Selection	26
6.2	Spatially Varying Regularisation	31
7	Conclusion	33
7.1	Summary	33
7.2	Future Work and Possible Extensions	33
A	Full Results Table	36

Acknowledgements

I would like to express my gratitude to Dr Kostas Papafitsoros for supervising me this semester and introducing me to the fascinating field of variational image analysis. His guidance, teaching and advice, both on the project itself and on wider academic matters, were invaluable throughout, and this project would not have been possible without his support.

1 Introduction and Background

When taking an image on a modern digital camera, we often take the clear, true-to-life result for granted. What we do not realise is the sophisticated mathematics occurring in the background to produce such results. Images acquired in real world settings are frequently corrupted by noise, blurring and other degradations arising from imperfect sensing conditions, low lighting, or camera acquisition error. Removing such degradations while preserving important features of an image is therefore a fundamental property of image processing.

One of the most widely studied methods of image denoising is the Total Variation[ROF92] method of denoising, or TV denoising for short. This method is a variational image denoising method that aims to create a suitable reconstruction of our image, that tries to strike a balance between removing as much noise as is possible and preserving image features. Within this method we have a regularisation parameter α that controls this balance between noise removal and feature preservation. The standard method for computing this parameter relies on supervised learning where we define a loss function, that is a function of our noisy image and the true image, and aim to seek the α that minimises this loss. Whilst this is an effective method the key drawback is this heavy reliance on ground truth images, making this method unsuitable for many real life scenarios where image denoising is required.

The solution: self-supervised approaches. A possible solution to this issue is the use of a self-supervised loss rather than a supervised loss. The key difference here is that a self-supervised loss does not require the ground truth data to determine the loss. Once we have this loss we can then proceed as per the supervised case and seek the α that minimises this loss.

In this project we explore several self-supervised loss functions, each with their own advantages and disadvantages. These include the Recorrupted2Recorrupted loss, the Neighbour2Neighbour loss, and the SURE(Stein’s Unbiased Risk Estimate) loss[TD26, HLJ⁺21]. We then explore how effective these self-supervised losses are at approximating the supervised loss using classic image analysis metrics such as PSNR(Peak Signal to Noise Ratio) and the SSIM(Structural Similarity Index). However before we can dive into this, we must first formalise some notions within image analysis so that we can better understand our problem.

In section 1 we begin this process and start with defining what inverse image problems and image denoising is and also explain concepts in variational image denoising. We also review some existing literature to see what current methods have found and how our proposed findings may fit into existing research. In section 2 we recall the standard supervised approach in order to establish a baseline for which we can compare our findings against. In section 3 we finally begin to explore our self-supervised losses starting with the Noise2Noise loss, which forms the bedrock of some of our other self-supervised losses. Sections 4 and 5 are dedicated to the computational side of the project as well as the spatially varying side, where we cover the practical implementation of our loss functions, as well as the implementation of our algorithms to solve our problem. The remaining sections focus on evaluating our experiments and discussing the results of our project.

1.1 Inverse Image Problems and Image Denoising

Let us first define what it means for an image to be a function. Consider a typical greyscale digital image of $N \times M$ resolution, i.e., it is made up of $N \cdot M$ pixels. We can represent this image as a function

$$u : \{1, \dots, N\} \times \{1, \dots, M\} \rightarrow [0, 1], \quad (i, j) \mapsto u(i, j)$$

where each pixel is mapped to some value in the interval $[0, 1]$. Within this convention we would have $u(i, j) = 0$ corresponding to a totally black pixel and $u(i, j) = 1$ corresponding to a completely white pixel. Anything else in between would be a shade of grey. For the purposes of our project we will only be using greyscale images, however all the methods we cover can be adapted for RGB images using the following definition of u ,

$$u : \{1, \dots, N\} \times \{1, \dots, M\} \rightarrow \{0, \dots, 255\}^3$$

We must also note that whilst images can in principle be defined as continuous functions on $\mathbb{R}^2$ or $\mathbb{R}^3$, for the purposes of this project we work exclusively in the discrete setting. For the remainder of this project we move to the vectorised representation $u \in \mathbb{R}^n$ where $n = N \cdot M$, which is standard in the inverse image problems literature. We note that whilst mathematically we treat images as vectors $u \in \mathbb{R}^n$, in practise our implementation operates on 2D arrays of shape $N \times M$, which is equivalent up to some reshaping.

Now that we have defined what a digital image is, we can now move onto the concept of an inverse image problem. Let us imagine that there exists some perfect image acquisition device that is capable of capturing a perfect measurement of our image $u \in \mathbb{R}^n$. We call this device a forward linear operator

$$A : \mathbb{R}^n \rightarrow \mathbb{R}^m$$

where m denotes the number of measurements. Intuitively we can think of this forward linear operator A as being our camera. In a perfect world where this "camera" is as good as can be and we have no external conditions or any random noise we would denote our observed image as the following

$$y = A(u)$$

However, in reality there is always random noise and possibly other features such as blurring caused by external impacts. Hence our measurement is better defined as the following

$$y = A(u) + \varepsilon$$

where our measurement y is made up of two parts. The first is $A(u)$ which is the version of the true image u captured during the acquisition process, along with ε which accounts for a random noise component, which follows a certain distribution, due to real world errors. In most cases solving an inverse image problem pertains to finding a reconstruction function $f : \mathbb{R}^m \rightarrow \mathbb{R}^n$ such that $f(y) \approx u$, i.e., the reconstructed image $f(y)$ is as close to the true image u as possible. The difficulty of this task is heavily dependant

on our operator A , that is responsible for modelling everything deterministic that the "camera" does to the image before random noise is added. As a result, different operators can massively impact the difficulty of our inverse problem. In this project we focus solely on the case of image denoising. This is a special case where our operator A is simply taken to be the identity operator: $A(x) = x$. In other words, the case of image denoising is essentially like having an excellent camera that is capable of capturing a perfect measurement of the true image u only to be corrupted by some random noise ε . This leads us to the following formulation of our measurement

$$y = u + \varepsilon$$

which we will stick with for the rest of this project. Image denoising is simply the process where we wish to find a suitable reconstruction of u that is as close to y as possible, with a minimal amount of noise.

At this stage one might ask why we cannot just invert the operator A in order to solve the inverse problem. There are two main reasons for why this is generally not possible. Firstly, the operator A may not be full rank, meaning there are infinitely many images u consistent with the same measurement y . Secondly, even when A is invertible, directly applying A^{-1} to y may amplify the noise dramatically, making a reconstruction far from the true image u . Even in the denoising case where $A = I$, whilst the first issue does not arise, the second is still very much present. Directly using y as our reconstruction is still unsatisfactory as it contains the noise ε . As a result we cannot rely on the inversion of A as a valid solution to our problem, motivating the need for more advanced approaches which we cover in the next section.

1.2 The Variational Approach

The Variational Approach provides us with a clean solution to the issues that arise when we try to simply invert our operator A . This is achieved through reformulating our inverse problem into a regularisation problem. Rather than trying to recover u via an inversion, we instead try to recover u as a minimiser of a function $\mathcal{J}$ which has the following form:

$$\mathcal{J}(u) = \Phi(Au, y) + \alpha\Psi(u)$$

We call the function Φ the fidelity term which is responsible for measuring the distance between the measurement y and the reconstruction u after the forward operator A has been applied to u . If $\Phi(Au, y)$ is small then it ensures the transformed image Au is close to the actual measurement y . We call Ψ the regulariser or regularisation term. It is responsible for imposing an extra layer of regularity upon u . Here $\alpha > 0$ is the regularisation parameter, which controls the balance between the fidelity and regularisation term. The choice of this parameter is crucial. A large α places more weight on the regularisation term, producing smoother reconstructions, whilst a small α places more weight on the fidelity term, staying closer to the noisy measurement. The central focus on this project is the problem of selecting an appropriate α without access to the clean image u . Intuitively speaking the fidelity term is responsible for making sure that our reconstruction stays as close to the measurement as possible whilst the regularisation term is responsible for penalising undesirable properties of the reconstruction such as excessive noise or unwanted oscillations. The process of minimising $\mathcal{J}$ is therefore finding

a suitable candidate for our image u such that it strikes a balance between remaining faithful to the noisy observation but also producing a clean well structured reconstruction.

The choice of function for Φ depends on our noise assumption, for example in the case of isotropic Gaussian noise, which is what we will be working with for this project, we define Φ to be the squared L^2 norm of the difference between Au and y :

$$\Phi(Au, y) = \frac{1}{2} \|Au - y\|^2$$

Different noise assumptions will therefore lead to different definitions, i.e., for poisson noise we use the K-L divergence between Au and y while for uniform noise we use the sup norm of the difference between Au and y . On the other hand the choice of Ψ is determined by our prior knowledge and assumptions of the properties of the desired image u . The choice of regulariser can often change the outcome of the reconstruction.

1.3 Total Variation Denoising

There are many possible choices for the regularisation term but as stated in the previous section, these choices can have a significant impact on the outcome of the reconstruction. For example a popular choice for Ψ is the *Tikhonov* regulariser [Tik63, Whi22] defined as so:

$$\Psi(u) = \|\nabla u\|_2^2$$

Whilst the Tikhonov regulariser is a classical well studied approach it does come with its drawbacks. As discussed in [Pap15] its use is known to result in reconstructions where sharp edges are over-smoothed, making it subpar for natural images. This motivates the need for other choices of the regulariser which are capable of removing noise but also maintaining these distinct image features such as sharp edges. A prominent choice of regulariser that achieves this, and the regulariser that we will focus on for the rest of this project, is the *Total Variation* regulariser, or TV regulariser for short. However before we formally define the TV regulariser we must define what it means to take a derivative on an image.

Again consider our image $u \in \mathbb{R}^n$. We define the horizontal derivative of u at a given pixel (i, j) to be

$$(\nabla_x u)_{i,j} = u_{i+1,j} - u_{i,j}$$

and the vertical derivative of u at a given pixel (i, j) to be

$$(\nabla_y u)_{i,j} = u_{i,j+1} - u_{i,j}$$

So the gradient of u at a pixel (i, j) is defined to be

$$(\nabla u)_{i,j} = \begin{pmatrix} u_{i+1,j} - u_{i,j} \\ u_{i,j+1} - u_{i,j} \end{pmatrix}$$

We define the Total Variation of u , pioneered in [ROF92], to be the sum of the euclidean magnitude of the gradient vectors of u :

$$\text{TV}(u) = \sum_{i,j} \sqrt{(u_{i+1,j} - u_{i,j})^2 + (u_{i,j+1} - u_{i,j})^2}$$

At this point we note that an anisotropic variant of TV exists which uses absolute values rather than euclidean magnitudes, however in this project we use isotropic formulation as defined above.

In a nutshell, Total Variation measures how much an image changes across space. In this discrete setting we are working in, this corresponds to an ℓ_1 -type penalty on finite differences between neighbouring pixels. The main advantage of this, as opposed to an alternative method such as the Tikhonov regulariser, is that it allows piecewise constant images to preserve their edges whilst also reducing noise.

To understand why TV regularisation is effective for variational image denoising, we must consider the total variation of a noisy image versus the total variation of a clean image. First let us consider the case of TV for a clean image. If we zoom into a small patch of the image and take a look at the gradient at each pixel we get the following:

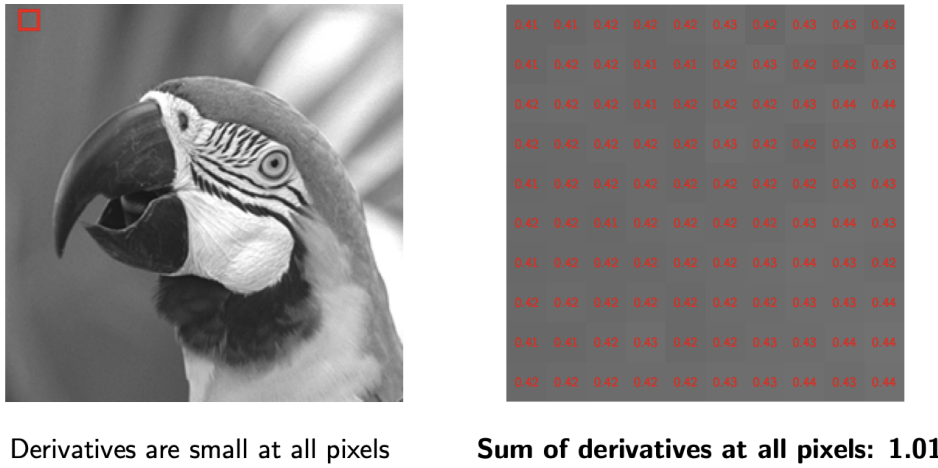
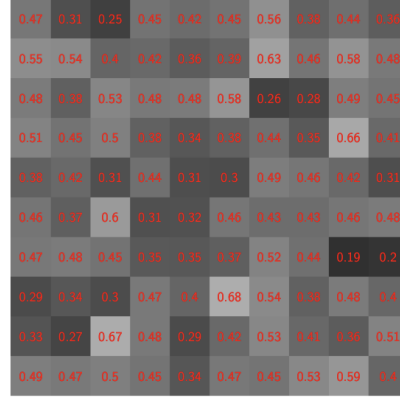


Figure 1: Gradient magnitudes in a patch of a clean image. Figure courtesy of Dr Kostas Papafitsoros.

As seen in Figure 1 when we have a clean image, the derivative at each pixel is very small. This is due to the fact that the image is smooth and there is no random noise, so there are no random fluctuations between pixels, meaning that pixel to pixel change is very small. This results in small gradient magnitudes across the whole image, leading to a small total variation. On the other hand if we consider the case of a noisy image:



noisy



Sum of derivatives at all pixels: 19.71

Figure 2: Gradient magnitudes in a patch of a noisy image. Figure courtesy of Dr Kostas Papafitsoros.

we immediately see the contrast compared to Figure 1. In the case of the noisy image, due to this addition of Gaussian noise, the noise adds random(independent) fluctuations at each pixel. As a result, even in flat areas, the pixel to pixel change is very large. Therefore we have large derivatives across the whole image resulting in many non zero gradients, giving us a total variation that is significantly larger than the total variation of the clean image.

This reveals a key property of Total Variation. Noisy images intrinsically have a much higher total variation than their clean counterparts. This is because the addition of noise introduces random, independent fluctuations at every pixel, resulting in large gradient magnitudes across the entire image even in regions that should be flat or smooth. A clean image on the other hand has large gradients only at genuine edges, with near zero gradients in flatter areas, resulting in a significantly lower total variation.

This observation is precisely what makes the TV regulariser so effective for image denoising. By introducing $TV(u)$ as our regularisation term Ψ we are penalising reconstructions with high total variation, implicitly discouraging the random pixel fluctuations that are a prominent trait of noise. Combined with the fidelity term, which ensures the reconstruction stays close to the noisy observation y , this gives us the TV denoising problem:

$$u^* = \arg \min_{u \in \mathbb{R}^n} \frac{1}{2} \|u - y\|^2 + \alpha TV(u)$$

Where the choice of this regularisation parameter α is of utmost importance. If α is too small then the solution of our minimisation problem will just tend towards the noisy image. On the other hand if α is too large, then whilst much of the noise is removed, the reconstruction will be over-smoothed and important image features such as edges will be lost. As a result, our problem boils down to trying to choose the alpha such that we strike a balance between staying faithful to the noisy image, but also removing as much noise as possible. This problem of choosing α is precisely what we will be focusing on for this project. In particular we are interested in self-supervised approaches to this problem, where we select α without access to the clean image u .

1.4 Review of Existing Literature

The problem of selecting an optimal regularisation parameter for TV denoising has been heavily studied in recent years. A prominent example is the work of Kofler et al. [KAAB⁺23], which proposes a deep unrolling framework for learning spatially varying regularisation parameter maps directly from data. Rather than estimating a single scalar parameter α for each image individually, the framework trains a deep neural network to learn a mapping from noisy input data to a spatially varying parameter map Λ . A second subnetwork then unrolls a fixed number of iterations of a TV minimisation algorithm incorporating this parameter map. The key advantage of this approach is that once trained, the network generalises to unseen images, avoiding the need to re-estimate parameters from scratch for each new image. This work is then extended by Vu et al. [VKP25] where the same technique is applied to the case of TGV denoising. The benefit of this is that all the advantages acquired from [KAAB⁺23] are now combined with the advantages of TGV denoising. So not only do we gain a parameter map rather than a point estimate, giving us a network that generalises well to unseen noisy images, but we also get all the benefits from TGV; namely the elimination of the stair casing effect, better feature preservation, and even better modelling of real world structures. In addition the paper also discovers some intrinsic properties of the parameter map, described as a *triple edge structure*. The authors found that network actually learns to apply different amounts of first order and second order regularisation depending on the structure of the image at that location. In other words the network learns how to adapt between TV-like and TGV-like behaviour depending on the region.

These two papers introduce some very useful techniques to the area, but have one common pitfall. They both rely on supervised learning. Both papers use a deep neural network to learn this parameter map, which involves using many ground truth images within the training process. As a result if we want these techniques to work for a group of images that do not have a clean counterpart, they may not work as intended. This limitation is what motivates us to investigate self-supervised learning techniques for computation of regularisation parameters, with the hope that some of the methods we explore could be applied to the above papers and enable them to work for cases where we don't have access to the clean reference images. We note that whilst the above methods employ deep neural networks, classical iterative solvers such as the Chambolle-Pock algorithm provide an equally valid framework for solving the underlying variational problems, and form the basis of our implementation.

2 The Supervised Approach

2.1 Parameter Selection via Supervised Losses

Now that we have formally defined the method of Total Variation Denoising, we can now begin to describe our supervised problem in more detail. Consider a clean image $u \in \mathbb{R}^n$ and a noisy observation of this image $y = u + \varepsilon$, where $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$. We define the supervised loss to be the following:

$$\mathcal{L}_{sup}(y, u, \alpha) = \|f_{\alpha}(y) - u_{true}\|^2$$

We note that from this section onwards we denote the true clean image as $u_{true} \in \mathbb{R}^n$ to distinguish it from the optimisation variable u which appears in the variational formulation. This loss along with our variational TV denoising formulation gives us the following bi-level optimisation problem:

$$\begin{cases} \alpha^* = \arg \min_{\alpha} \|f_{\alpha}(y) - u_{true}\|^2 \\ f_{\alpha}(y) = \arg \min_{u \in \mathbb{R}^n} \frac{1}{2} \|u - y\|^2 + \alpha \text{TV}(u) \end{cases}$$

The above bi-level optimisation problem is the standard formulation used in supervised parameter selection for TV denoising. Here $f_{\alpha}(y)$ denotes the TV denoised reconstruction of y at a regularisation parameter α , obtained by solving the inner optimisation problem. The goal of this bi-level problem is to try and find the optimum α that minimises the supervised loss. This is because, by comparing our reconstruction (at a particular α) to the clean reference image u , we are able to minimise the reconstruction error, allowing us to generate a reconstruction that is as close to the clean image as possible. Despite the fact that this supervised formulation of our problem is widely used it comes with one major drawback; the supervised loss function is a function of u_{true} .

In many real world applications this clean image u is often expensive or in some cases even impossible to obtain. For example in real-world imaging settings, where the noisy observation of the image is all we have access to, this method immediately fails because a clean counterpart of our image does not even exist. This makes the supervised approach impractical in such settings, which motivates the objective of our project. We aim to investigate the use of select self-supervised loss functions, that allow us to alter our optimisation problem so that they can select α using only the noisy observation y .

3 Self-Supervised Approaches

The propositions and proofs presented in this section are based on those found in [TD26], [PZQJ21] and [HLJ⁺21]. We have expanded them to include additional intermediate steps and detail for clarity, however the original results and ideas are due to the respective authors.

3.1 Overall Strategy

We now turn our attention to the central contribution of our project where we define and explore the self-supervised methods we focus on. The main idea here is that we aim to find a suitable replacement for our supervised loss function that does not require us to have the clean image in advance. Let us recall our formulation of our bi-level optimisation problem:

$$\begin{cases} \alpha^* = \arg \min_{\alpha} \|f_{\alpha}(y) - u_{true}\|^2 \\ f_{\alpha}(y) = \arg \min_{u \in \mathbb{R}^n} \frac{1}{2} \|u - y\|^2 + \alpha \text{TV}(u) \end{cases}$$

To be more formal, the goal is to replace the outer layer of our optimisation problem, the part that is minimising over α , and make it so that it does not depend on the clean image, u_{true} . An important property that we aim for our self-supervised loss to have is

the property of being an unbiased estimator. In other words an ideal self-supervised loss $\mathcal{L}(y, \alpha)$ should satisfy the following:

$$\mathbb{E}[\mathcal{L}(y, \alpha)] = \mathbb{E}[\|f_\alpha(y) - u_{true}\|^2] + \text{const}$$

where the expectation is taken over the noise distribution $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$ with u_{true} fixed.

Whilst this may be a simple problem to state, in practise this can become quite complicated especially when we are trying to make this self-supervised loss as accurate as possible. In the following sections we introduce three self-supervised losses: the *recorrup2recorrup2* loss, the *neighbour2neighbour* loss, and the *SURE* loss. In some cases such as the SURE loss, we will see that we can achieve the stronger result of the exact equality without the constant, meaning the self-supervised loss also provides an exact estimate of the reconstruction error.

3.2 The Foundation: Noise2Noise Loss

Before we define the Recorrup2Recorrup2 loss, it is important to first understand the Noise2Noise loss, as this self-supervised loss provides the motivation for both the Recorrup2recorrup2 loss and the Neighbour2Neighbour loss.

The Noise2Noise loss, originally proposed by Lehtinen et al. [LMH⁺18], considers the case where we are able to observe two independent observations of our image u_{true} . In other words instead of just having one measurement $y = u_{true} + \varepsilon$, we instead have $y_1 = u_{true} + \varepsilon_1$ and $y_2 = u_{true} + \varepsilon_2$ where $\varepsilon_1, \varepsilon_2 \sim \mathcal{N}(0, \sigma^2 I)$ are independent. We can then define our loss to be the following:

$$\mathcal{L}_{noise2noise}(y_1, y_2, \alpha) = \|f_\alpha(y_1) - y_2\|^2$$

which as you can see does not depend on the clean image u_{true} . The key idea behind this is as follows. Rather than trying to use the distance between the reconstruction and our clean image, let us instead take two (independent) observations of our clean image, reconstruct one of them, and then find the difference between those two. The hope is that, on average, the two losses will be equivalent up to the addition of some constant, that is independent of both the clean image u_{true} and the reconstruction f_α . Meaning that minimising over α is unaffected by the constant. Let us now formalise this intuition with the following result.

Proposition 1 (Tachella et al. [TD26]). *Let y_1 and y_2 be two random variables independent conditional on u_{true} , and assume that $\mathbb{E}_{y_2|u_{true}}[y_2] = u_{true}$. Then:*

$$\mathbb{E}_{y_1, y_2|u_{true}}[\|f_\alpha(y_1) - y_2\|^2] = \mathbb{E}_{y_1|u_{true}}[\|f_\alpha(y_1) - u_{true}\|^2] + \text{const}$$

where the constant is independent of f_α .

Proof.

$$\begin{aligned}
& \mathbb{E}_{y_1, y_2 | u_{true}} [\|f_\alpha(y_1) - y_2\|^2] \\
&= \mathbb{E}_{y_1, y_2 | u_{true}} [\|(f_\alpha(y_1) - u_{true}) - (y_2 - u_{true})\|^2] \\
&= \mathbb{E}_{y_1 | u_{true}} [\|f_\alpha(y_1) - u_{true}\|^2] - 2\mathbb{E}_{y_1, y_2 | u_{true}} [(f_\alpha(y_1) - u_{true})^\top (y_2 - u_{true})] + \mathbb{E}_{y_2 | u_{true}} [\|y_2 - u_{true}\|^2] \\
&= \mathbb{E}_{y_1 | u_{true}} [\|f_\alpha(y_1) - u_{true}\|^2] - 2\mathbb{E}_{y_1, y_2 | u_{true}} [(f_\alpha(y_1) - u_{true})^\top (y_2 - u_{true})] + \text{const} \\
&= \mathbb{E}_{y_1 | u_{true}} [\|f_\alpha(y_1) - u_{true}\|^2] - 2\mathbb{E}_{y_1 | u_{true}} [f_\alpha(y_1) - u_{true}]^\top \mathbb{E}_{y_2 | u_{true}} [y_2 - u_{true}] + \text{const} \\
&= \mathbb{E}_{y_1 | u_{true}} [\|f_\alpha(y_1) - u_{true}\|^2] - 2\mathbb{E}_{y_1 | u_{true}} [f_\alpha(y_1) - u_{true}]^\top (u_{true} - u_{true}) + \text{const} \\
&= \mathbb{E}_{y_1 | u_{true}} [\|f_\alpha(y_1) - u_{true}\|^2] + \text{const}
\end{aligned}$$

where the fifth line uses the conditional independence of y_1 and $y_2 | u_{true}$ to factorise the expectation, and the sixth line uses the assumption $\mathbb{E}_{y_2 | u_{true}} [y_2] = u_{true}$ which gives $\mathbb{E}_{y_2 | u_{true}} [y_2 - u_{true}] = u_{true} - u_{true} = 0$. $\square$

This result shows us that the Noise2Noise loss gives us an unbiased estimate of the supervised loss. Whilst this is a good result, we can only apply it when we have two independent observations of the noisy image y , which cannot be achieved in many real life applications. The following two sections are dedicated to exploring loss functions that bypass this hurdle by artificially creating the two independent observations of the noisy image, whilst still using the underlying philosophy of the Noise2Noise method.

3.3 Recorruped2Recorruped Loss

The Recorruped2Recorruped loss provides us with the first of our two possible methods to bypass the independence condition placed on the two observations of u_{true} .

Let us consider the following set up. Given some image $u_{true} \in \mathbb{R}^n$, we acquire a measurement of our image $y = u_{true} + \varepsilon$ where $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$. Unlike Noise2Noise, in this case we cannot assume that we have two independent observations y_1 and y_2 . We can, however, proceed in the case described in [PZQJ21] and create the two independent realisations y_1 and y_2 by adding synthetic noise to our noisy image y . We do this by resampling two independent noisy realisations of the original measurement y like so:

$$\begin{cases} y_1 = y + \sqrt{\frac{\beta}{1-\beta}} \omega \\ y_2 = y - \sqrt{\frac{1-\beta}{\beta}} \omega \end{cases}$$

where $\omega \sim \mathcal{N}(0, \sigma^2 I)$ follows the same distribution as the noise ε and $\beta \in (0, 1)$ is a positive scalar parameter. The idea behind this is that we create our own independent observations and if they are indeed independent then we can just proceed as per the case of the Noise2Noise loss and create a loss function that is also an unbiased estimator of the supervised loss, but without requiring two independent observations. The key concept that this hinges on however, is whether these two synthetic re-corruptions y_1 and y_2 are actually conditionally independent on the image u_{true} ; since this is the crucial requirement for the Noise2Noise argument to be applied. We clarify this with the following result below:

Proposition 2 (Pang et al. [PZQJ21]). *The random variables y_1 and y_2 defined as above, are independent conditional on u_{true} for any $\beta \in (0, 1)$.*

Proof. Let $\tau = \sqrt{\frac{\beta}{1-\beta}}$. We can then rewrite y_1 and y_2 as:

$$y_1 = u_{true} + \varepsilon + \tau\omega, \quad y_2 = u_{true} + \varepsilon - \frac{1}{\tau}\omega$$

Since y_1 and y_2 are linear combinations of Gaussian random variables, they are jointly Gaussian conditional on u_{true} . For jointly Gaussian random variables, zero cross-covariance is equivalent to independence. It therefore suffices to show that $\text{Cov}(y_1, y_2 | u_{true}) = 0$.

We first compute the conditional expectations:

$$\begin{aligned} \mathbb{E}_{y_1|u_{true}}[y_1] &= \mathbb{E}_{\varepsilon,\omega}[u_{true} + \varepsilon + \tau\omega] = u_{true} + \mathbb{E}_{\varepsilon}[\varepsilon] + \tau\mathbb{E}_{\omega}[\omega] = u_{true} \\ \mathbb{E}_{y_2|u_{true}}[y_2] &= \mathbb{E}_{\varepsilon,\omega}\left[u_{true} + \varepsilon - \frac{1}{\tau}\omega\right] = u_{true} + \mathbb{E}_{\varepsilon}[\varepsilon] - \frac{1}{\tau}\mathbb{E}_{\omega}[\omega] = u_{true} \end{aligned}$$

where both noise terms vanish since $\mathbb{E}[\varepsilon] = \mathbb{E}[\omega] = 0$. By the definition of cross-covariance:

$$\text{Cov}(y_1, y_2 | u_{true}) = \mathbb{E}_{y_1, y_2 | u_{true}} [(y_1 - u_{true})(y_2 - u_{true})^\top]$$

Substituting $y_1 - u_{true} = \varepsilon + \tau\omega$ and $y_2 - u_{true} = \varepsilon - \frac{1}{\tau}\omega$:

$$\begin{aligned} &= \mathbb{E}_{\varepsilon,\omega} \left[(\varepsilon + \tau\omega) \left(\varepsilon - \frac{1}{\tau}\omega \right)^\top \right] \\ &= \mathbb{E}_{\varepsilon,\omega} \left[\varepsilon\varepsilon^\top - \frac{1}{\tau}\varepsilon\omega^\top + \tau\omega\varepsilon^\top - \omega\omega^\top \right] \\ &= \mathbb{E}_{\varepsilon}[\varepsilon\varepsilon^\top] - \frac{1}{\tau}\mathbb{E}_{\varepsilon}[\varepsilon]\mathbb{E}_{\omega}[\omega^\top] + \tau\mathbb{E}_{\omega}[\omega]\mathbb{E}_{\varepsilon}[\varepsilon^\top] - \mathbb{E}_{\omega}[\omega\omega^\top] \\ &= \mathbb{E}_{\varepsilon}[\varepsilon\varepsilon^\top] - \mathbb{E}_{\omega}[\omega\omega^\top] \end{aligned}$$

where the third line uses linearity of expectation and independence of ε and ω , and the fourth line uses $\mathbb{E}[\varepsilon] = \mathbb{E}[\omega] = 0$ to eliminate the middle two terms. Since $\omega \sim \mathcal{N}(0, \sigma^2 I)$ follows the same distribution as $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$:

$$\mathbb{E}_{\varepsilon}[\varepsilon\varepsilon^\top] = \mathbb{E}_{\omega}[\omega\omega^\top] = \sigma^2 I$$

Therefore:

$$\text{Cov}(y_1, y_2 | u_{true}) = \sigma^2 I - \sigma^2 I = 0$$

Since y_1 and y_2 are jointly Gaussian conditional on u_{true} and have zero cross-covariance, they are conditionally independent given u_{true} . $\square$

Now that we have established that our synthetic observations y_1 and y_2 are indeed conditionally independent given u_{true} , we can define the Recorruted2Recorruted loss (R2R loss) as follows:

$$\mathcal{L}_{R2R}(y_1, y_2, \alpha) = \mathbb{E}_{y_1, y_2 | y} [\|f_\alpha(y_1) - y_2\|^2]$$

and because y_1 and y_2 are conditionally independent and $\mathbb{E}_{y_2|u_{true}}[y_2] = u_{true}$ it is an unbiased estimator of the supervised loss. We prove this equivalence below:

Proof. We want to show that:

$$\mathbb{E}_{y|u_{true}} [\mathcal{L}_{R2R}] = \mathbb{E}_{y_1|u_{true}} [\|f(y_1) - u_{true}\|^2] + \text{const}$$

We start with the expectation of the R2R loss:

$$\mathbb{E}_{y|u_{true}} [\mathcal{L}_{R2R}] = \mathbb{E}_{y|u_{true}} [\mathbb{E}_{y_1, y_2|y} [\|f_\alpha(y_1) - y_2\|^2]]$$

By the law of iterated expectations we know that

$$\mathbb{E}_X [\mathbb{E}_{Y|X} [g(X, Y)]] = \mathbb{E}_{X, Y} [g(X, Y)]$$

Hence:

$$\mathbb{E}_{y|u_{true}} [\mathbb{E}_{y_1, y_2|y} [\|f_\alpha(y_1) - y_2\|^2]] = \mathbb{E}_{y_1, y_2|u_{true}} [\|f_\alpha(y_1) - y_2\|^2]$$

and by our proof in section 3.2, the above expression is an unbiased estimator of the supervised loss. $\square$

Whilst R2R successfully bypasses the need for two independent observations, it still requires knowledge of the noise distribution σ^2 to generate the synthetic observations. In the following section we introduce the Neighbour2Neighbour loss which addresses this limitation.

3.4 Neighbour2Neighbour Loss

The Neighbour2Neighbour loss [HLJ⁺21] offers a second method for generating independent observations from a single noisy image, without requiring knowledge of the noise distribution. This is achieved through the method of sub-sampling. Rather than trying to synthetically create two conditionally independent observations of our noisy image y , we instead use a subsampling procedure (described in more detail below) to generate noisy image pair $(g_1(y), g_2(y))$ from a single noisy image y . At this point one might ask if we can take this subsampled pair and directly substitute them into our Noise2Noise formula like we did in the R2R case:

$$\mathbb{E} [\|f_\alpha(g_1(y)) - g_2(y)\|^2]$$

This, however, will not work. Unlike R2R where y_1 and y_2 are made to be independent observations of the same underlying image u_{true} , the subsampled images $(g_1(y), g_2(y))$ have slightly different ground truths. This is because the subsampling procedure constructs the pairs using slightly different pixels. As a result directly applying the Noise2Noise loss in this setting can result in over smoothing. To correct for this [HLJ⁺21] introduces an additional regularisation term that accounts for this gap, allowing us to define the Neighbour2Neighbour loss as follows:

$$\mathcal{L}_{N2N} = \mathbb{E} [\|f_\alpha(g_1(y)) - g_2(y)\|^2 + \gamma \|f_\alpha(g_1(y)) - g_2(y) - g_1(f_\alpha(y)) + g_2(f_\alpha(y))\|^2]$$

where $\gamma > 0$ is a regularisation parameter that controls the strength of the regularisation term. Please note that in this project our main focus is on the computation of α and so therefore we fix $\gamma = 0.5$, though different values may yield different results. A key advantage of the Neighbour2Neighbour loss over both R2R and SURE is that it makes no

assumptions about the noise distribution. Unlike R2R which requires knowledge of σ^2 to generate synthetic observations, and SURE which requires σ^2 explicitly in the correction term, the N2N loss depends only on the noisy observation y and the subsampling procedure, making it applicable in settings where the noise level is unknown. At this stage it is important to note that unlike the Noise2Noise and R2R losses, the N2N loss does not provide us with an unbiased estimator of the supervised loss. As outlined in [HLJ⁺21], the N2N loss is only approximately unbiased. The approximation becomes exact as the gap $\varepsilon = \mathbb{E}[g_2(y)] - \mathbb{E}[g_1(y)] \rightarrow 0$. The regularisation term in the loss is responsible for correcting this gap, and so in practise the approximation becomes reasonable.

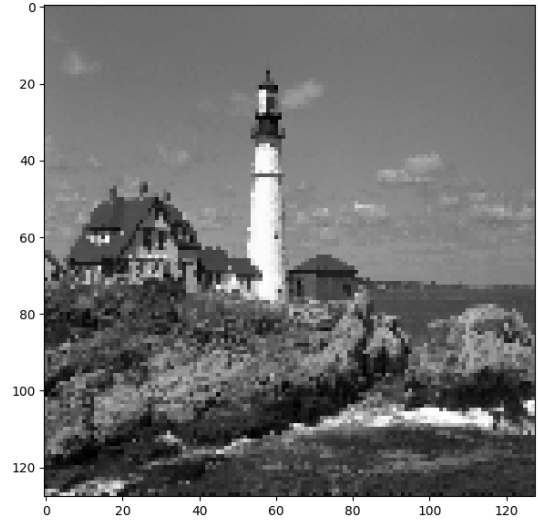
We now describe the subsampling procedure to show how the pair $(g_1(y), g_2(y))$ are generated. Let us assume our image y has width W and height H .

- Step 1.** Divide the image y into $\lfloor W/K \rfloor \times \lfloor H/K \rfloor$ sub images, where each sub image is of size $K \times K$. In our implementation we set $K = 2$
- Step 2.** For the i th row and j th column cell, two neighbouring locations are randomly selected. They are respectively taken as the (i,j) -th element of the sub sampler $G = (g_1, g_2)$.
- Step 3.** For all the $\lfloor W/K \rfloor \times \lfloor H/K \rfloor$ sub images, repeat step 2. Then the neighbour sub sampler $G = (g_1, g_2)$ is generated.

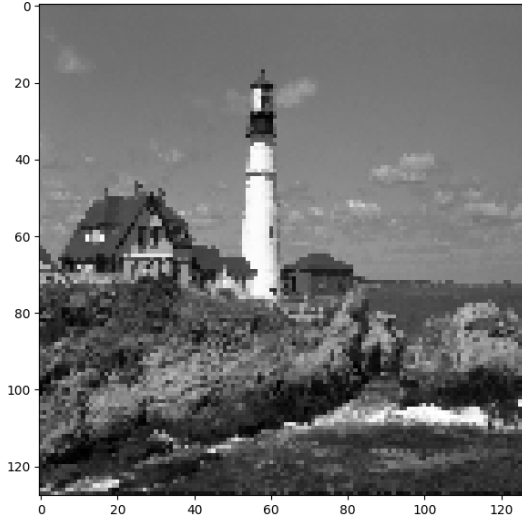
A practical example of what the sub sampling procedure does to an image can be seen below:



(a) Original image y



(b) Subsampled image $g_1(y)$



(c) Subsampled image $g_2(y)$

Figure 3: Illustration of the Neighbour2Neighbour subsampling procedure. The original image, in this case a clean image u_{true} , is subsampled into two half-resolution images $g_1(y)$ and $g_2(y)$, both of which still resemble the original image despite being at lower resolution.

Having now explored two self-supervised losses that bypass the independence requirement of Noise2Noise through generating additional artificial observations, we now turn to a third approach: the *SURE* loss.

3.5 SURE Loss

The *Stein's Unbiased Risk Estimate (SURE)* loss takes an entirely different approach to our two previous losses. Rather than artificially generating additional observations of y we instead start with a naive version of the self-supervised loss, and proceed to then add additional terms to correct the error caused by the initial naive loss. Let us begin by recalling the supervised loss, which is what we want to be able to approximate. Please also note that throughout this section all expectations are taken with u_{true} fixed, i.e. over the noise distribution $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$.

$$\mathcal{L}_{sup}(y, u_{true}, \alpha) = \frac{1}{n} \|f_\alpha(y) - u_{true}\|^2$$

Unlike the R2R and N2N losses, we retain the factor of $\frac{1}{n}$ throughout this section. This normalises the loss by the total number of pixels n , ensuring scale invariance with respect to image size, and yields a cleaner final expression for the SURE loss, in particular the noise constant term simplifies to σ^2 rather than $n\sigma^2$.

If we rewrite $f_\alpha(y) - u_{true} = (f_\alpha(y) - y) + (y - u_{true})$ and then expand the supervised loss, we get the following:

$$\frac{1}{n} \|f_\alpha(y) - u_{true}\|^2 = \frac{1}{n} \|f_\alpha(y) - y\|^2 + \frac{2}{n} (f_\alpha(y) - y)^\top (y - u_{true}) + \frac{1}{n} \|y - u_{true}\|^2$$

Taking expectations on both sides gives us:

$$\mathbb{E} \left[\frac{1}{n} \|f_\alpha(y) - u_{true}\|^2 \right] = \mathbb{E} \left[\frac{1}{n} \|f_\alpha(y) - y\|^2 \right] + \frac{2}{n} \mathbb{E} [(f_\alpha(y) - y)^\top (y - u_{true})] + \sigma^2$$

After re-arranging for the naive loss we achieve the following equality:

$$\mathbb{E} \left[\frac{1}{n} \|f_\alpha(y) - y\|^2 \right] = \mathbb{E} \left[\frac{1}{n} \|f_\alpha(y) - u_{true}\|^2 \right] - \frac{2}{n} \mathbb{E} [(f_\alpha(y) - y)^\top (y - u_{true})] - \sigma^2$$

This shows us that simply trying to use the naive loss as our self-supervised loss is a biased estimator of the supervised loss. Moreover, it tells us that it differs by two correction terms; one of which involves the clean image u_{true} which is unknown: $\frac{2}{n} \mathbb{E} [(f_\alpha(y) - y)^\top (y - u_{true})]$. And the other involves the known constant σ^2 . This tells us that if we can find a way to replace the correction term involving u_{true} , then we can generate an unbiased estimate of the supervised loss that does not rely on the Noise2Noise method. In order to do this we must introduce the following proposition:

Proposition 3 (Stein [Ste81]). *Let $y|u_{true} \sim \mathcal{N}(u_{true}, \Sigma)$ and f_α be weakly differentiable. Then:*

$$\mathbb{E} [f_\alpha(y)^\top (y - u_{true})] = \mathbb{E} \left[\text{trace} \left(\frac{\partial f_\alpha}{\partial y} \Sigma \right) \right]$$

where $\frac{\partial f_\alpha}{\partial y} \in \mathbb{R}^{n \times n}$ is the Jacobian of f_α at y .

With Stein's lemma in hand we can now tackle this correction term. Consider the

cross-term:

$$\begin{aligned}
(f_\alpha(y) - y)^\top (y - u_{true}) &= (f_\alpha(y) - u_{true} + u_{true} - y)^\top (y - u_{true}) \\
&\text{adding and subtracting } u_{true} \text{ gives us:} \\
&(f_\alpha(y) - u_{true})^\top (y - u_{true}) + (u_{true} - y)^\top (y - u_{true}) \\
&\text{but } (u_{true} - y) = -(y - u_{true}) \text{ so} \\
&= (f_\alpha(y) - u_{true})^\top (y - u_{true}) - (y - u_{true})^\top (y - u_{true}) \\
&= (f_\alpha(y) - u_{true})^\top (y - u_{true}) - \|y - u_{true}\|^2
\end{aligned}$$

Substituting this relation into our above equality gives us the following:

$$\mathbb{E} \left[\frac{1}{n} \|f_\alpha(y) - y\|^2 \right] = \mathbb{E} \left[\frac{1}{n} \|f_\alpha(y) - u_{true}\|^2 \right] - \frac{2}{n} \mathbb{E} [(f_\alpha(y) - u_{true})^\top (y - u_{true})] - \sigma^2$$

and by linearity of expectation we then arrive at the following equality:

$$\begin{aligned}
\mathbb{E} \left[\frac{1}{n} \|f_\alpha(y) - y\|^2 \right] &= \mathbb{E} \left[\frac{1}{n} \|f_\alpha(y) - u_{true}\|^2 \right] - \frac{2}{n} \mathbb{E} [(f_\alpha(y) - u_{true})^\top (y - u_{true})] + \frac{2}{n} \mathbb{E} [\|y - u_{true}\|^2] \\
&\quad - \sigma^2 \\
&= \mathbb{E} \left[\frac{1}{n} \|f_\alpha(y) - u_{true}\|^2 \right] - \frac{2}{n} [\mathbb{E} [f_\alpha(y)^\top (y - u_{true})] - \mathbb{E} [u_{true}^\top (y - u_{true})]] \\
&\quad + \frac{2}{n} (\sigma^2 \cdot n) - \sigma^2 \\
&= \mathbb{E} \left[\frac{1}{n} \|f_\alpha(y) - u_{true}\|^2 \right] - \frac{2}{n} [\mathbb{E} [f_\alpha(y)^\top (y - u_{true})] - u_{true}^\top \mathbb{E} [y - u_{true}]] + \sigma^2 \\
&= \mathbb{E} \left[\frac{1}{n} \|f_\alpha(y) - u_{true}\|^2 \right] - \frac{2}{n} [\mathbb{E} [f_\alpha(y)^\top (y - u_{true})] - u_{true}^\top \cdot 0] + \sigma^2 \\
&= \mathbb{E} \left[\frac{1}{n} \|f_\alpha(y) - u_{true}\|^2 \right] - \frac{2}{n} \mathbb{E} [f_\alpha(y)^\top (y - u_{true})] + \sigma^2 \\
&\quad \text{directly applying Stein's lemma then gives us} \\
&= \mathbb{E} \left[\frac{1}{n} \|f_\alpha(y) - u_{true}\|^2 \right] - \frac{2}{n} \mathbb{E} \left[\text{trace} \left(\frac{\partial f_\alpha}{\partial y} \Sigma \right) \right] + \sigma^2
\end{aligned}$$

Using this relation we can now define the SURE loss in such a way that it will be an unbiased estimator of the supervised loss. Rearranging, we can now define the SURE loss as:

$$\mathcal{L}_{SURE}(y, \alpha) = \frac{1}{n} \|f_\alpha(y) - y\|^2 + \frac{2}{n} \text{trace} \left(\frac{\partial f_\alpha}{\partial y} \Sigma \right) - \frac{1}{n} \text{trace}(\Sigma)$$

which by construction satisfies $\mathbb{E}[\mathcal{L}_{SURE}] = \mathbb{E}[\mathcal{L}_{SUP}]$. In our case $\text{trace}(\Sigma) = n \cdot \sigma^2$, giving us the following simplified formulation of the SURE loss:

$$\mathcal{L}_{SURE}(y, \alpha) = \frac{1}{n} \|f_\alpha(y) - y\|^2 + \frac{2}{n} \text{trace} \left(\frac{\partial f_\alpha}{\partial y} \Sigma \right) - \sigma^2$$

There is actually some nice intuition behind this SURE loss. Unlike R2R and N2N which generate additional observations to bypass the need for the clean image, SURE takes a

different approach. Instead it estimates the true reconstruction error by correcting the naive residual loss using the sensitivity of the denoiser to small perturbations of the noisy input. The key quantity driving this correction is the divergence term, which measures how strongly the denoiser reacts to changes in the noisy image. A large divergence indicates that the estimator is closely tracking the noise, i.e. fitting random fluctuations rather than the underlying signal. A small divergence indicates the estimator is suppressing noise, producing smoother reconstructions. SURE uses this sensitivity as a correction to the naive residual. If the denoiser reacts strongly to noisy pixels, SURE increases the estimated error, penalising this behaviour and pushing towards a better regularised solution. In this way SURE strikes a balance between the naive residual estimate and the true reconstruction error, achieving what R2R and N2N achieve through independent observations, but using only the single noisy image y and knowledge of the noise level σ^2

The only issue that remains is how to approximate this divergence term in practise. For much smaller images computing the full jacobian may be feasible, but for larger images (like the ones we are working with in our project) computing the full jacobian can be completely intractable. Since there are generally no analytic solutions for the divergence term in the SURE loss, we resort to Monte Carlo estimates. Two approaches are available:

Method 1 (Automatic Differentiation): Using Hutchinson’s unbiased trace estimator [Hut89] with $\omega \sim \mathcal{N}(0, \Sigma)$:

$$\text{trace} \left(\Sigma \frac{\partial f_\alpha}{\partial y} \right) \approx \omega^\top \frac{\partial f_\alpha}{\partial y} \omega$$

This approach computes the Jacobian-vector product $\frac{\partial f_\alpha}{\partial y} \omega$ exactly via automatic differentiation, without forming the full Jacobian matrix.

Method 2 (Finite Difference): Alternatively, using a Monte Carlo finite difference approximation [RBU08] with $\omega \sim \mathcal{N}(0, I)$:

$$\text{trace} \left(\Sigma \frac{\partial f_\alpha}{\partial y} \right) \approx \left(\frac{\Sigma \omega}{\tau} \right)^\top (f_\alpha(y + \tau \omega) - f_\alpha(y))$$

where $\tau > 0$ is a small perturbation parameter. This approximation becomes exact as $\tau \rightarrow 0$ in expectation over ω . In our case where $\Sigma = \sigma^2 I$ this simplifies to:

$$\approx \frac{\sigma^2}{\tau} \omega^\top (f_\alpha(y + \tau \omega) - f_\alpha(y))$$

In this project we choose to use Method 2 since our TV denoising solver involves operations and iterative computations that may not be differentiable. As outlined in [TD26], in practice τ is chosen to be a small fraction of the noise standard deviation σ , and a single sample of ω is sufficient due to the spatial averaging over n pixels.

4 The Case of The Spatially Varying Regularisation Parameter

4.1 The Main Idea

In this section we explore the case of TV denoising with the spatially varying regularisation parameter. The main difference of this approach is that rather than using one global scalar parameter α that is applied to every single pixel in the image, we instead assign different regularisation strengths to different regions. This is because most natural images are not the same throughout space. They typically contain both smooth regions, such as flat backgrounds or open skies, as well as regions with fine detail and sharp edges, such as textures and object boundaries. As a result using one global regularisation parameter α may not always be the best, since an α sufficiently large enough will inevitably also over smooth fine details and blur edges. On the other hand switching to the spatially varying regularisation parameter Λ allows us to apply more regularisation in the smoother areas whilst simultaneously applying less regularisation in areas with finer details and edges. This spatial adaptivity results in better preservation of important image features and in many cases can result in a reconstruction with higher quality than the global approach. In our project, we want to explore whether this observation also holds for self-supervised methods, more specifically the SURE loss.

4.2 Practical Implementation

In practise we take a block based approach where we divide the image into 256, 16×16 blocks and find the optimal α_k for each of these $k = 1, 2, \dots, 256$ blocks. The collection of these 256 α_k 's is then what gives us our spatially varying regularisation parameter $\Lambda = \{\alpha_k\}_{k=1}^{256}$. In the supervised case, the computation of the α_k for each of the blocks is done using the same approach as the global α . Essentially we treat each of these sub-blocks as if they are a separate image, and find the α_k that solves the supervised bi-level optimisation problem for block k . Once all 256 α_k 's have been found, we apply each α_k to its corresponding patch to obtain the denoised patch $f_{\alpha_k}(y_k)$. Finally, these denoised patches are reassembled to form the full spatially varying reconstruction. In the self-supervised case the only difference is that we minimise the self-supervised loss rather than the supervised loss when trying to find the optimal α_k .

To illustrate the benefit of the spatially varying approach, we present a visual comparison between the global and spatially varying supervised reconstructions for a single test image in Figure 4. The alpha heatmap reveals that the spatially varying approach assigns higher regularisation values to smooth regions such as the sky, whilst assigning lower values to areas with finer detail such as the rocks/water and the lighthouse. This results in a PSNR improvement of 0.40 dB over the global supervised approach, demonstrating that even a simple block based spatially varying scheme can yield meaningful gains in reconstruction quality.

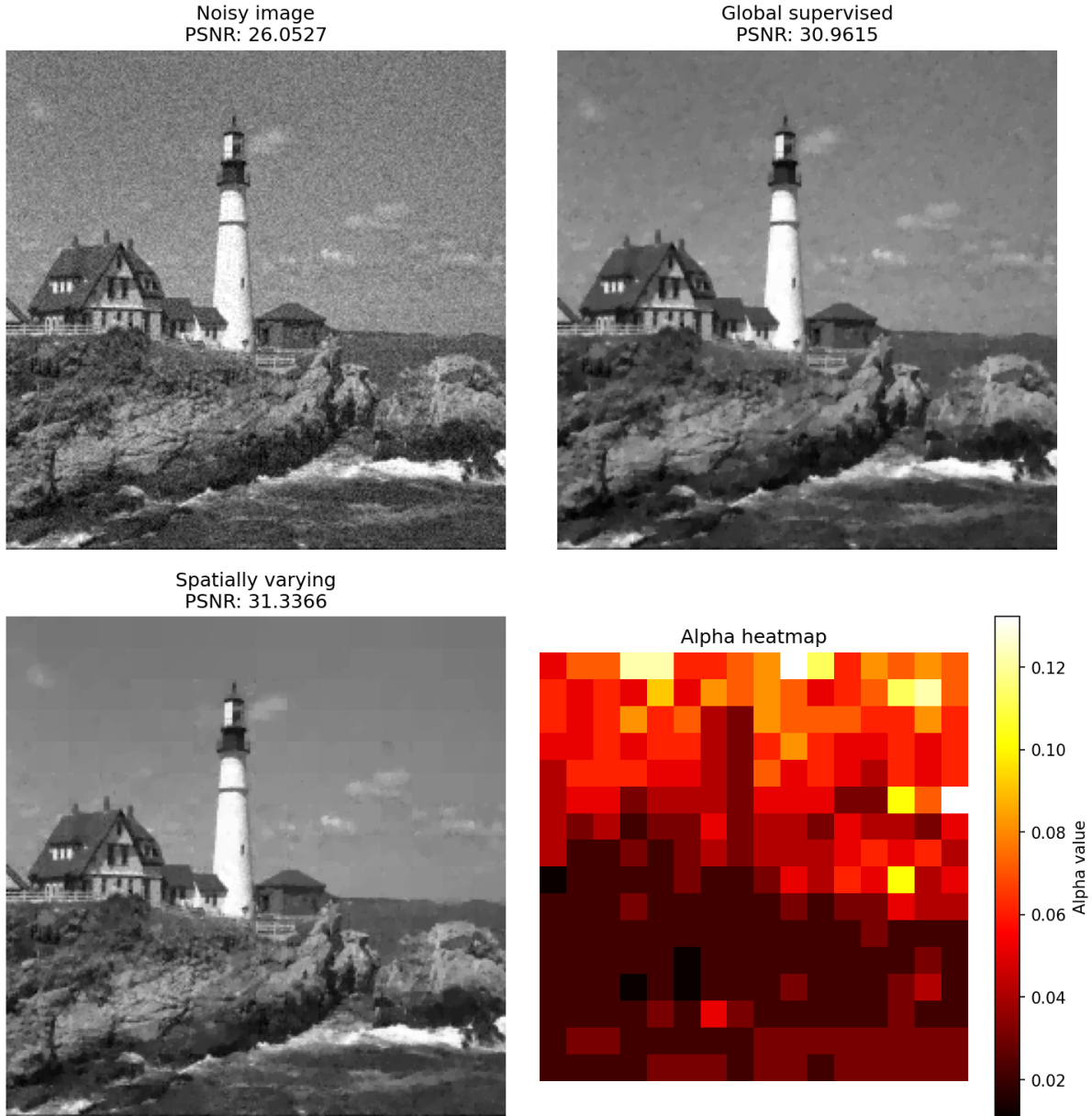


Figure 4: Comparison of spatially varying approach vs global approach

5 Implementation

5.1 Data

For this project the dataset we used for all testing and evaluation purposes was the Kodak dataset [Com93]. The Kodak dataset is a collection of 24 high quality colour images. The original images all either have a resolution of 768 x 512 pixels or 512 x 768 pixels. For the purposes of this project all these images were resized to be 256 x 256 pixels, converted to greyscale and normalised so that all pixel values are within $[0, 1]$.

5.2 Chambolle-Pock Algorithm

The Chambolle-Pock algorithm [CP11] is a first order primal-dual algorithm designed for solving convex optimisation problems of the following form:

$$\min_u F(Ku) + G(u)$$

where F and G are convex functions and K is a linear operator. This makes it particularly well suited for solving the TV denoising problem, where the non smooth nature of the TV term makes standard gradient descent methods inadequate. The algorithm works by alternating between a dual update, a primal update and an extrapolation step at each iteration:

$$\begin{aligned} p^{k+1} &= \text{prox}_{\sigma F^*} (p^k + \sigma K \bar{u}^k) \\ u^{k+1} &= \text{prox}_{\tau G} (u^k - \tau K^\top p^{k+1}) \\ \bar{u}^{k+1} &= 2u^{k+1} - u^k \end{aligned}$$

where τ and σ are step size parameters satisfying $\tau\sigma\|K\|^2 \leq 1$, and prox denotes the proximal operator. The algorithm is guaranteed to converge to the global minimum under mild conditions [CP11]. In our implementation we run the algorithm for 1000 iterations which we found to be sufficient for convergence. We note that the practical implementation of this algorithm was kindly provided by Dr Kostas Papafitsoros.

5.3 Algorithm Implementation

Before we explore the practical implementations of our loss self-supervised loss functions we must establish some details about our images. All images we work with are resized to be 256 x 256 pixels, converted to greyscale and normalised so that all pixel values are within $[0, 1]$. In addition when we artificially add noise to our images we use a noise level of $\sigma = 0.05$. We also set our random seed to be 0 for reproducibility purposes. The full implementation of all algorithms described in this project is available at: <https://github.com/RayyanHameed/ThirdYearProjectCode>. This includes implementations of the SURE, R2R and N2N loss functions, the Chambolle-Pock TV denoising solver, the global parameter selection procedure and the spatially varying parameter selection procedure. We note that throughout this section $f_\alpha(\cdot)$ refers to the practical implementation of the Chambolle-Pock solver, kindly provided by Dr Papafitsoros, evaluated at a given α using 1000 iterations.

We begin with the R2R loss which we implement as defined in Section 3.3. The R2R loss only takes two parameters, the noisy image y and the regularisation parameter α . As seen in Section 3.3, the generation of the synthetic observations y_1 and y_2 depends on the parameter $\beta \in (0, 1)$. Since our project solely focuses on the computation of the optimal α we fix beta to be 0.5. We then move onto computing the R2R loss. This involves taking the expectation of the loss. In practise this expectation (and the ones present in the N2N and SURE loss) cannot be computed analytically. Instead we approximate them using Monte Carlo estimation, replacing the expectation with an average over K independent samples. For the R2R loss, at each iteration we draw $\omega^{(k)} \sim \mathcal{N}(0, \sigma^2 I)$ and construct the re-corrupted pairs:

$$y_1^{(k)} = y + \tau\omega^{(k)}, \quad y_2^{(k)} = y - \frac{\omega^{(k)}}{\tau}$$

where $\sigma = 0.05$ is consistent with our noise level. The loss is then approximated as:

$$\mathcal{L}_{R2R} \approx \frac{1}{K} \sum_{k=1}^K \|f_{\alpha}(y_1^{(k)}) - y_2^{(k)}\|^2$$

By the law of large numbers this approximation converges to the true expectation as $K \rightarrow \infty$. During testing we originally opted for $K = 3$ draws but found the loss to have too much variance over different runs, prompting us to switch to $K = 10$ draws. Whilst this drastically reduced the variance it made the time taken to compute α significantly longer. As a result we opted to settle for $K = 7$ draws which provided a balance between these two cases.

For the N2N loss our practical implementation works by averaging over $K = 7$ random sub-samplings. This averaging is necessary because the N2N loss is defined as an expectation over the random subsampling procedure. Since this expectation cannot be computed analytically (like the R2R case), we approximate it by averaging over K independent random sub-samplings. Prior to the loop, $f_{\alpha}(y)$ is computed once for computational efficiency and reused at each iteration. At each iteration, the sub-sampler randomly generates the pair $(g_1(y), g_2(y))$ from the noisy image, and a corresponding pair $g_1(f_{\alpha}(y)), g_2(f_{\alpha}(y))$ from the reconstruction. The reconstruction loss ℓ_{rec} and the regularisation loss ℓ_{reg} are then computed and accumulated, with $\gamma = 0.5$ as explained in Section 3.4. The final loss is then the average over all K iterations.

Finally, we move on to the SURE loss. Like the R2R and N2N losses the SURE loss only takes the noisy image y and the regularisation parameter α as its inputs. Our practical implementation also requires Monte Carlo estimation as stated in Section 3.5, due to the use of Hutchinson’s trace estimator, which is a special type of Monte Carlo estimate. However, unlike R2R and N2N, as seen in [TD26], a single sample of ω is sufficient due to the spatial averaging over n pixels. As stated in Section 3.5 we opt to use the finite difference method, which requires two evaluations of the TV denoising solver. One for $f_{\alpha}(y)$ and one for the perturbed image $f_{\alpha}(y + \tau\omega)$. With this method we have the small perturbation parameter $\tau > 0$. We set τ to be one percent of the noise level, i.e. $\tau = 0.01 \times \sigma = 0.0005$, as suggested in [TD26]. The loss is then computed as:

$$\mathcal{L}_{SURE} = \frac{1}{n} \|f_{\alpha}(y) - y\|^2 + \frac{2\sigma^2}{n\tau} \omega^{\top} (f_{\alpha}(y + \tau\omega) - f_{\alpha}(y)) - \sigma^2$$

We now move onto our chosen method for actually solving our bi-level optimisation problem. During the development phase we use two different methods for this. The first was a simple, but effective, brute force grid search method. With this method, the optimal α is selected through a grid search over a uniform grid of 350 points on the interval $[0.001, 0.2]$. We start by plotting a list of these 350 α points within the $[0.001, 0.2]$ interval. We then compute the loss at each of these 350 α ’s. to get a list of values for that given loss. Using these two lists we then plot the loss against the α ’s and select the optimal α from where the plot attains its minimum. The choice of a dense grid with 350 points was not entirely arbitrary. At first we started with only 100 points but found this to be insufficient. The smaller step size meant the grid was too coarse, causing methods with similar optimal parameters (particularly SURE and supervised) to snap to the same grid point and select identical values of α . This issue was a result of the discrete grid rather than a genuine

identical agreement between the two methods. Hence increasing the grid to 350 points and significantly reducing the step size was sufficient to distinguish between optimal parameters of different methods for all 24 test images. Whilst this method results in the most accurate computation of α it had a very long run time, prompting us to try another method.

The second method we tried during the development phase was the *Golden Section Search* method. We implemented this method as an alternative to the brute force grid search method. Golden Section Search is an efficient bracketing algorithm for finding the minimum of a unimodal function on a bounded interval, where we reduce the search interval by a factor of the golden ratio each iteration. This requires significantly fewer function evaluations than a dense grid search to locate the minimum. Whilst this approach did dramatically reduce the runtime (with the optimal α computed in under 30 seconds for the supervised loss), upon comparing the PSNR values obtained using alphas selected by the Golden Section Search against those found from the grid search, we noticed that Golden Section Search consistently underperformed. One possible reason for this could have been that Golden search operated under the assumption that the objective function is smooth and unimodal. However this is not always true for our self-supervised losses due to the added randomness from the Monte Carlo estimates. As a result we opted for the grid search approach for all experiments, accepting the longer run time in exchange for more reliable and accurate parameter selection.

5.4 Experimental Setup

For our experimental setup, we test all four losses (both supervised and self supervised) on all 24 images in our dataset. Again we note that we resize all images to 256 x 256 pixels, convert to greyscale and normalise so that all pixel values are within $[0, 1]$. We then add Gaussian noise to each image using a noise level of $\sigma = 0.05$. In order to solve the bilevel optimisation problem for each loss function we use our grid search approach for the outer layer (with 350 grid points) and our practical implementation of the Chambolle-Pock Primal Dual algorithm with 1000 iterations. After our plots of loss against alpha have been plotted and the α 's at which the minima have been attained are recorded we then evaluate the effectiveness of the reconstructions at these given α 's using two different metrics. The first of these is the Peak Signal to Noise Ratio (PSNR).

PSNR: Given two images $u_1, u_2 \in \mathbb{R}^{N \times M}$ with pixel values normalised to $[0, 1]$, the Mean Squared Error is defined as:

$$\text{MSE}(u_1, u_2) = \frac{1}{NM} \sum_{i=1}^N \sum_{j=1}^M (u_1(i, j) - u_2(i, j))^2$$

The Peak Signal to Noise Ratio is then defined as:

$$\text{PSNR}(u_1, u_2) = 10 \log_{10} \left(\frac{1}{\text{MSE}(u_1, u_2)} \right)$$

where the maximum pixel value is 1 since our images are normalised to $[0, 1]$. A higher PSNR indicates a better reconstruction.

The second metric we use is The Structural Similarity Index (SSIM). **SSIM:** The Structural Similarity Index [WBSS04] between two images u_1 and u_2 is defined as:

$$\text{SSIM}(u_1, u_2) = \frac{(2\mu_1\mu_2 + c_1)(2\sigma_{12} + c_2)}{(\mu_1^2 + \mu_2^2 + c_1)(\sigma_1^2 + \sigma_2^2 + c_2)}$$

where μ_1, μ_2 are the local means, σ_1^2, σ_2^2 are the local variances, σ_{12} is the local covariance of u_1 and u_2 , and c_1, c_2 are small stability constants. SSIM takes values in $[-1, 1]$ with 1 indicating perfect similarity. Unlike PSNR which measures pixel-wise error, SSIM captures perceptual similarity by also accounting for structural information, luminance and contrast. We note that in our analysis we primarily use the PSNR to evaluate how the self-supervised losses compare to our supervised baseline whilst the SSIM is used as a more complementary metric to provide a more complete picture of reconstruction quality.

6 Results

6.1 Global Parameter Selection

We present the results of our experiments on all 24 images from the Kodak dataset. For each image we plot the PSNR as a function of α for all four methods, with the selected optimal α for each method marked. We also present the corresponding reconstructions at these optimal α values. Figures 6 – 10 show these results for three representative images.

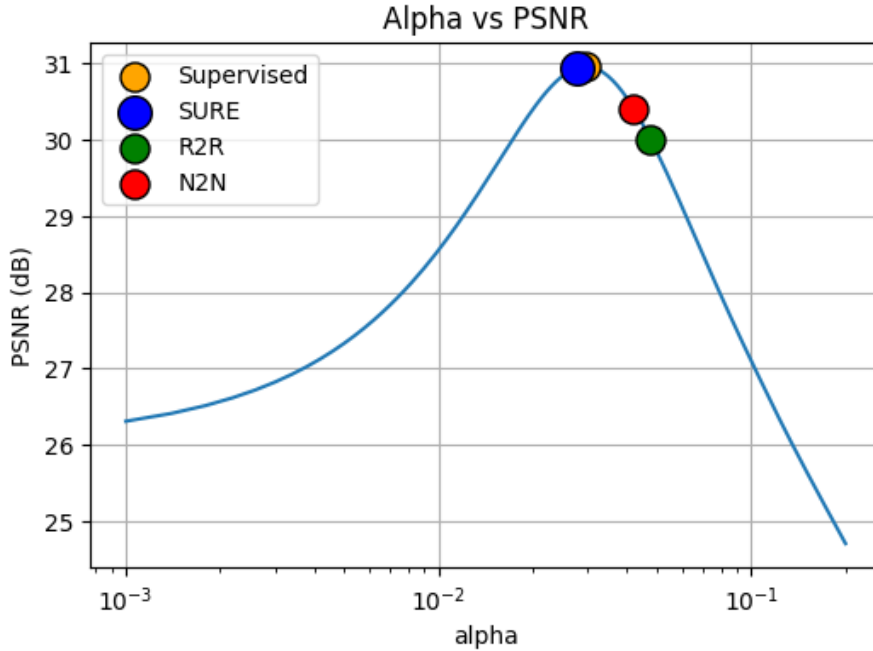


Figure 5: PSNR vs α for kodim21.

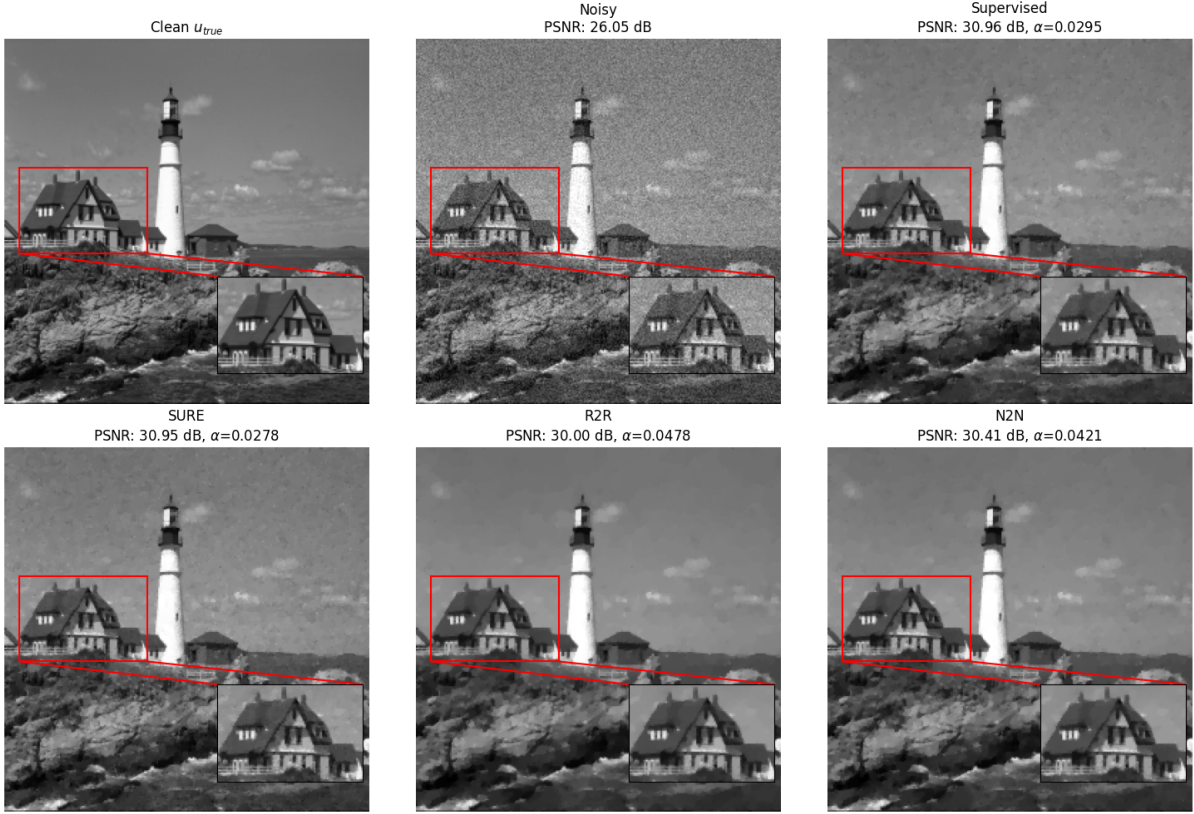


Figure 6: Reconstructions for kodim21. Top row: clean u_{true} , noisy y , supervised reconstruction. Bottom row: SURE, R2R and N2N reconstructions.

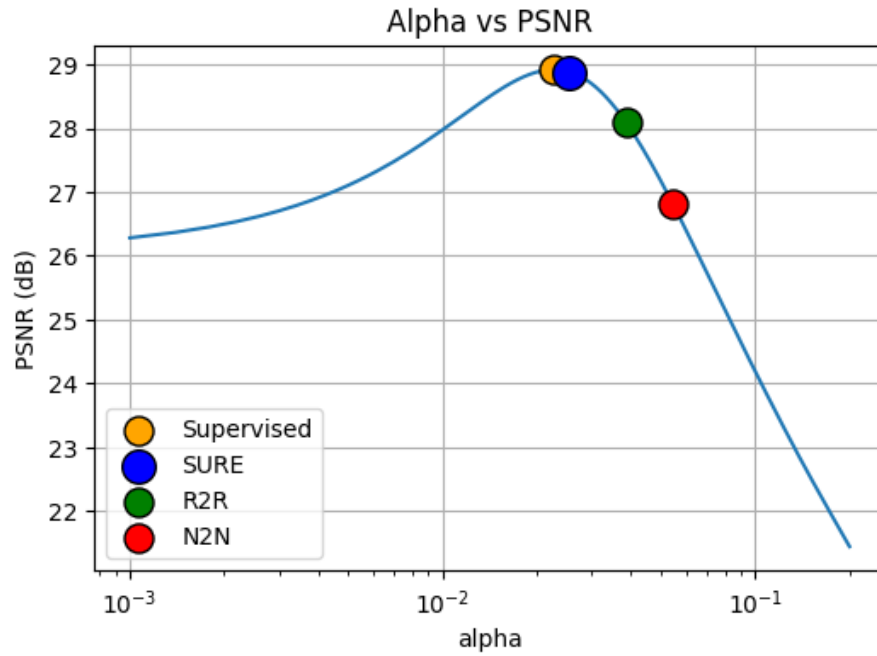


Figure 7: PSNR vs α for kodim05.

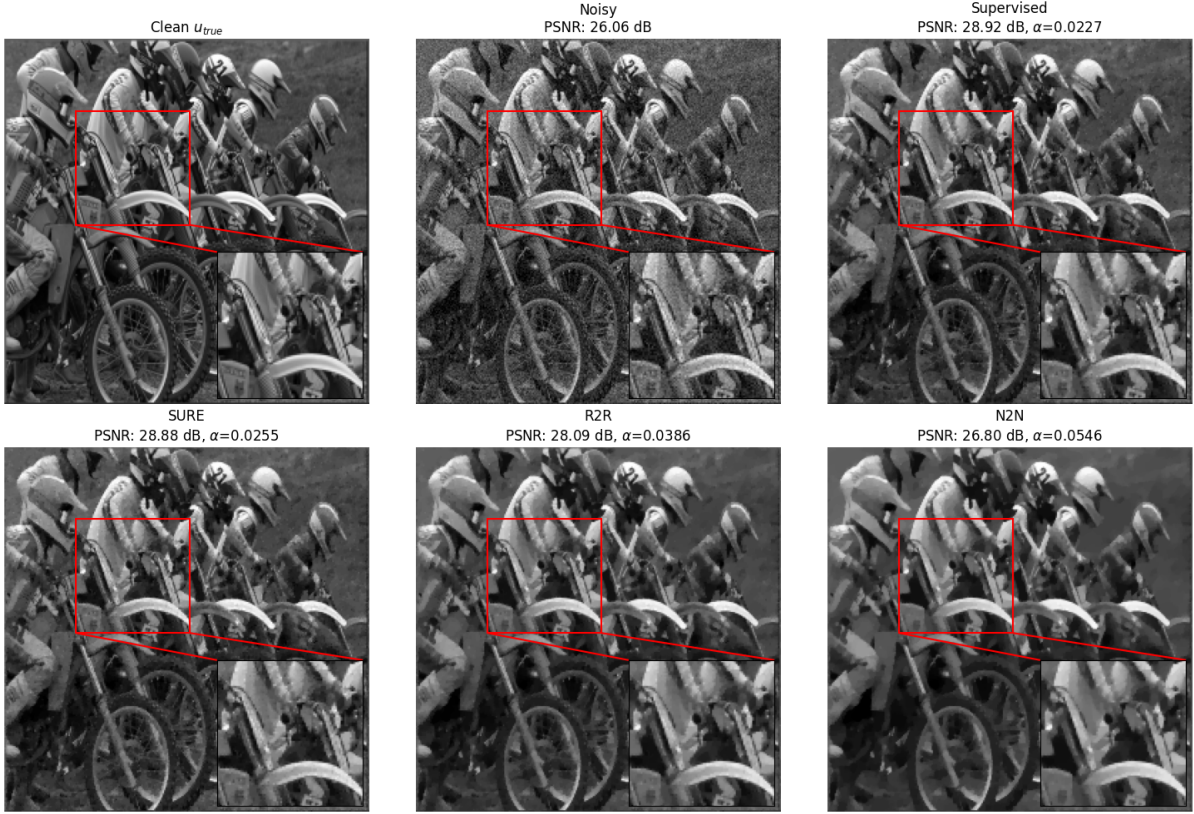


Figure 8: Reconstructions for kodim05. Top row: clean u_{true} , noisy y , supervised reconstruction. Bottom row: SURE, R2R and N2N reconstructions.

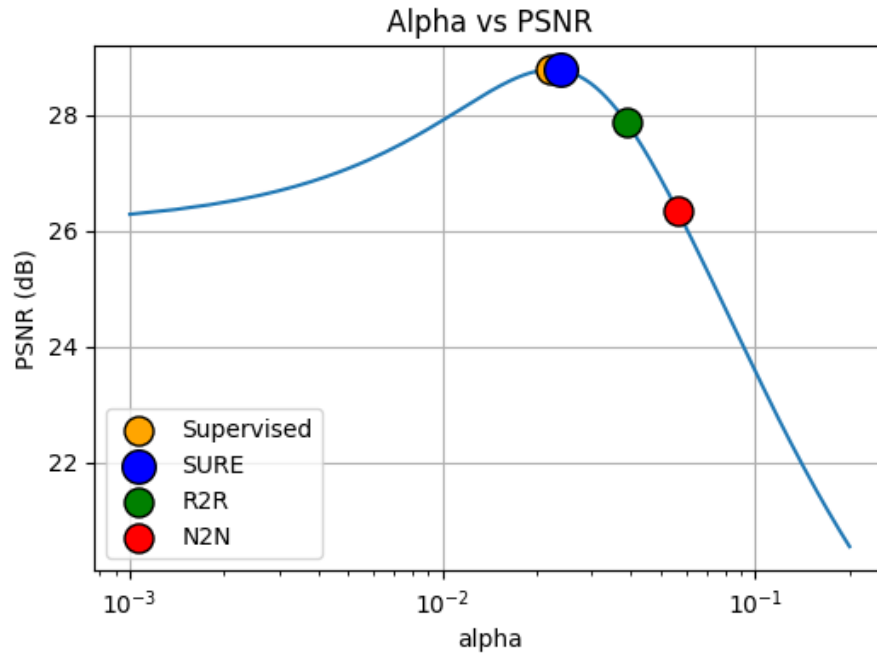


Figure 9: PSNR vs α for kodim08.

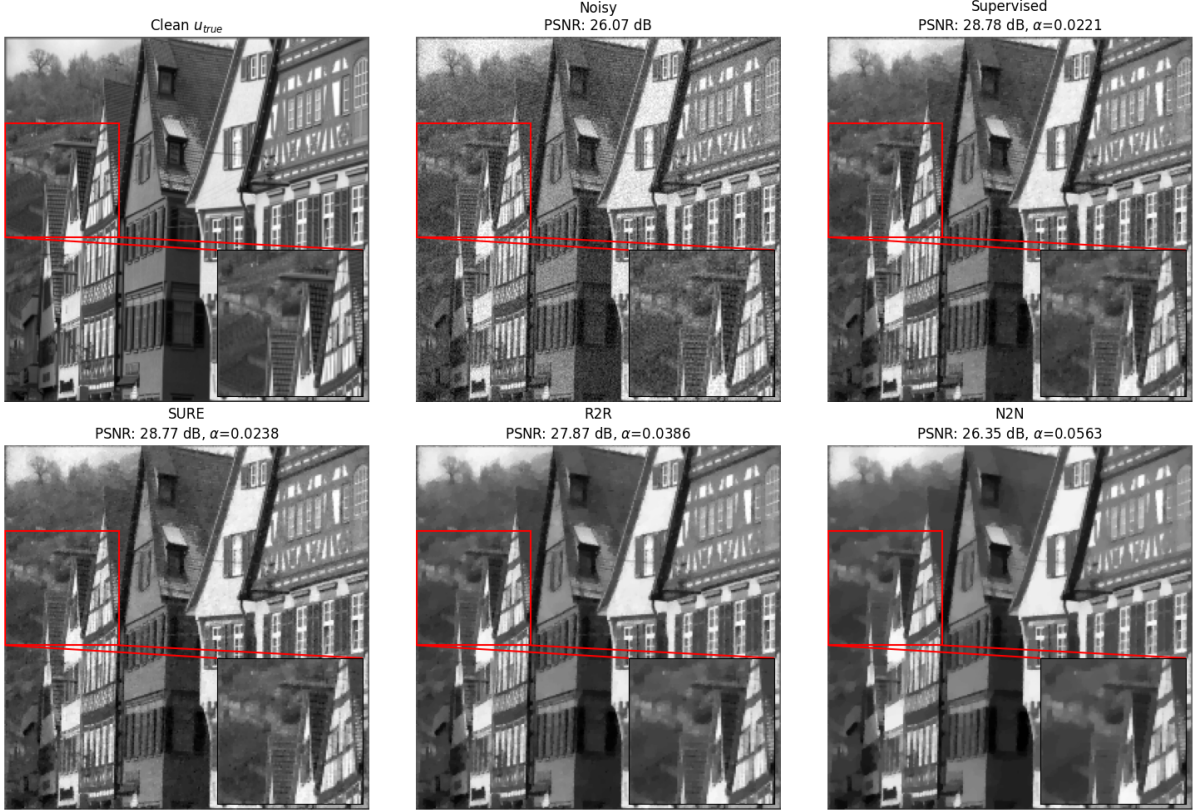


Figure 10: Reconstructions for kodim08. Top row: clean u_{true} , noisy y , supervised reconstruction. Bottom row: SURE, R2R and N2N reconstructions.

The above three cases give us a good view of how each of our self supervised methods perform generally. As seen in all three plots the SURE loss provides the best estimate of the supervised loss. For kodim05 and kodim08 the SURE loss slightly overestimates α , whilst for kodim21 it slightly underestimates α . In all three cases however the reconstruction generated from the SURE α has a PSNR that is extremely close to the supervised PSNR. This can be best seen in Figures 6, 8, 10, where its very difficult to see a visual difference between the supervised and SURE reconstructions. The R2R and N2N losses provide us with an overestimate of our regularisation parameter α to varying degrees. This is seen across all three test images, where we find that both the R2R and N2N losses overestimate the parameter α , resulting in a reconstruction with an overall lower PSNR when compared to the SURE and supervised losses. However this overestimation has varying degrees. In Figure 5 it is a relatively small overestimation resulting in a similar PSNR to the supervised reconstruction (30.00 and 30.41 respectively). On the other hand in Figures 7 and 9 this overestimate is much larger, especially for the N2N losses. The consequence of this can be best seen in Figure 10 where we can see the effects of the over smoothing. Not only does the reconstruction smooth-out the trees in the background but all the fine line detail has been removed from the roof of the house.

We now take a look at the results taken over all 24 images to see whether this trend with our losses is shared amongst the other images. Table 1 summarises the mean and standard deviation of PSNR and SSIM across all 24 images. Full results for all 24 images are provided in Table 2 in the Appendix.

Method	Mean PSNR (dB)	Std PSNR (dB)	Mean SSIM	Std SSIM
Noisy	26.1209	0.2281	0.6172	0.1019
Supervised	31.3996	1.5198	0.8681	0.0257
SURE	31.3757	1.5272	0.8666	0.0263
R2R	30.5293	1.5669	0.8491	0.0378
N2N	30.5711	2.1189	0.8466	0.0426

Table 1: Mean and standard deviation of PSNR and SSIM across all 24 Kodak images.

The summary statistics in Table 1 show that the trends observed in the three representative images generalise across the full Kodak dataset — SURE consistently achieves the closest performance to the supervised baseline, whilst R2R and N2N both overestimate α , resulting in lower mean PSNRs. More specifically, SURE achieves a mean PSNR of 31.3757 dB, within 0.024 dB of the supervised baseline of 31.3996 dB, demonstrating that it provides a highly effective self-supervised alternative to the supervised approach. R2R and N2N achieve mean PSNRs of 30.5293 dB and 30.5711 dB respectively, both lower than SURE but still significantly above the noisy baseline of 26.1209 dB. The standard deviation of N2N is notably higher than the other methods at 2.1189 dB, indicating less consistent performance across different images like in the case of the kodim08 image as seen in Figure 10.

We now take a look at the relative gaps between each of our self-supervised losses and our supervised baseline. Figure 11 shows the distribution of alpha gaps, defined as the supervised α minus the method α , across all 24 images for each self-supervised method.

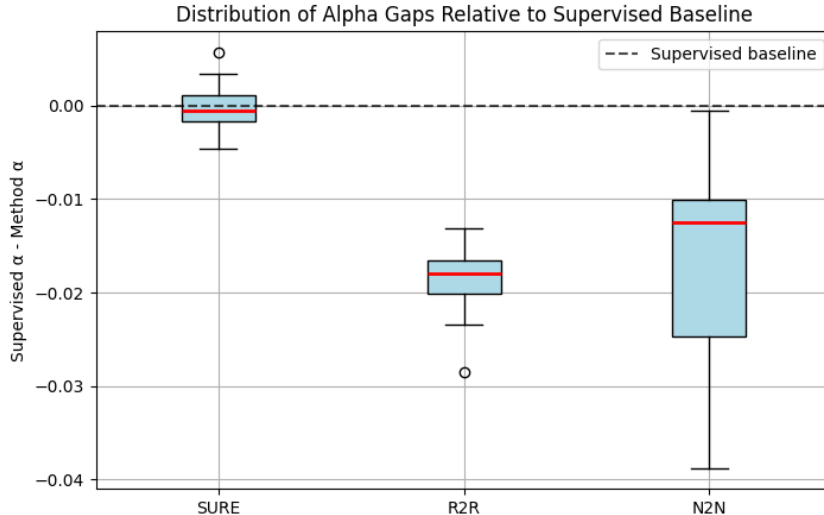


Figure 11: Distribution of alpha gaps (supervised α minus method α) across all 24 Kodak images. A value of zero indicates perfect agreement with the supervised baseline.

The box plot reveals that SURE consistently selects α values very close to the supervised baseline, with a median gap close to zero. R2R and N2N on the other hand tend to select larger α values than the supervised baseline, indicated by the negative

gaps. Overall this result in the R2R and N2N losses producing reconstructions with more aggressive regularisation and as a result, lower PSNR values.

6.2 Spatially Varying Regularisation

We now move onto our spatially varying approach, where we chose to focus on applying the SURE loss, as it had the best global performance, to the spatially varying case. We test both the spatially varying supervised and spatially varying SURE approaches on four representative images from the Kodak dataset: kodim01, kodim06, kodim15 and kodim21. For each image we compare the global supervised, spatially varying supervised and spatially varying SURE reconstructions in terms of PSNR to determine whether the SURE loss can produce similar reconstructions to the supervised loss.

After running the tests on all five images we obtain the following results:



Figure 12: Comparison of global supervised, spatially varying supervised, global SURE and spatially varying SURE reconstructions for kodim01.

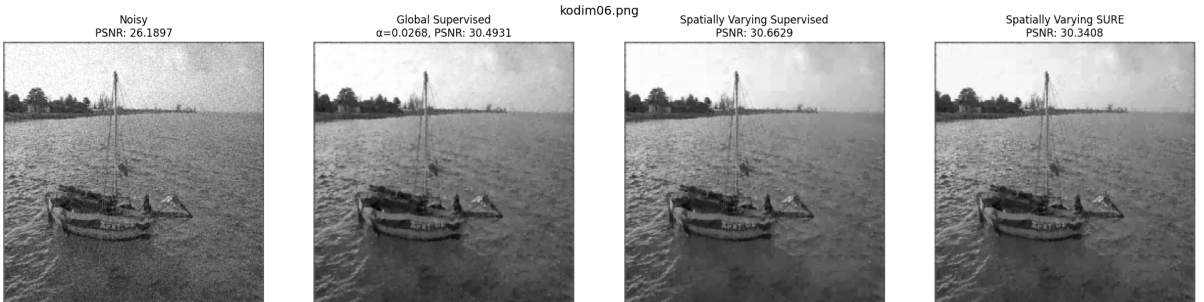


Figure 13: Comparison of global supervised, spatially varying supervised, global SURE and spatially varying SURE reconstructions for kodim06.

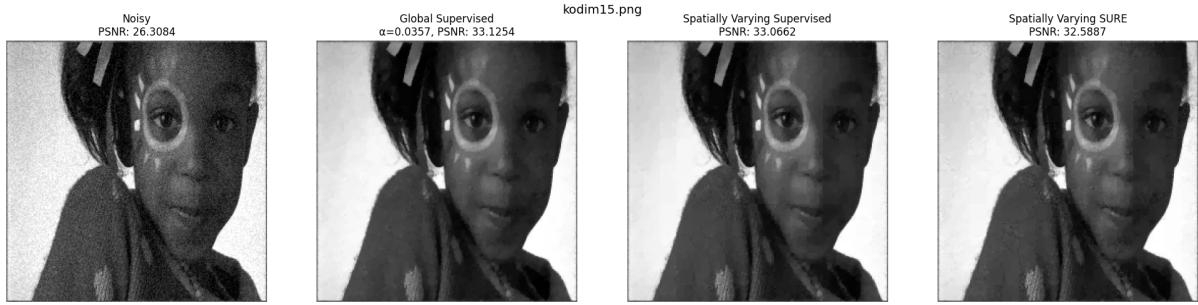


Figure 14: Comparison of global supervised, spatially varying supervised, global SURE and spatially varying SURE reconstructions for kodim015.

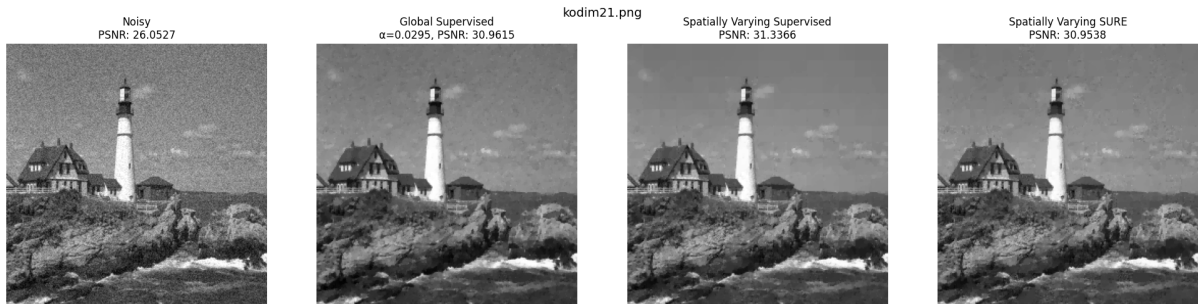


Figure 15: Comparison of global supervised, spatially varying supervised, global SURE and spatially varying SURE reconstructions for kodim21.



Figure 16: Comparison of global supervised, spatially varying supervised, global SURE and spatially varying SURE reconstructions for kodim23.

As we can see from the above figures, the spatially varying supervised approach consistently outperforms the global supervised baseline across all four images, confirming the benefit of spatial adaptivity in parameter selection. The spatially varying SURE approach however yields sub par results. For some images, as seen in Figures 15 and 13 it achieves a PSNR that is very close global supervised baseline. Whilst for others it falls below both the global supervised approach and the spatially varying supervised approach. We attribute this to the increased variance of the SURE divergence estimate at the block level, as discussed in Section 3.5. With only $16 \times 16 = 256$ pixels per block, the spatial averaging that makes SURE reliable at the full image scale is no longer sufficient, leading to unreliable alpha selection, and in turn leading to reconstructions with a lower PSNR. Interestingly, whilst the spatially varying SURE approach consistently underperforms in terms of PSNR, the visual differences between the reconstructions are subtle and

difficult to discern by eye, unlike our global supervised and self-supervised reconstructions where the differences are a lot more prominent. This highlights a potential issue in our use of PSNR as an evaluation metric. It measures pixel-wise error and does not always correlate well with visual image quality. Despite this numerical gap, the spatially varying SURE reconstructions are visually reasonable, suggesting that the approach may still have practical merit even if the PSNR does not fully capture this.

7 Conclusion

7.1 Summary

In this project, we investigated self-supervised methods for the computation of optimal regularisation parameters in Total Variational denoising. We investigated and implemented several self-supervised loss functions, namely the SURE, R2R and N2N losses [TD26], [HLJ⁺21], and evaluated their performance against the supervised baseline across 24 images from the Kodak dataset [Com93].

Our results display that the SURE loss provided a highly effective self-supervised alternative to the supervised approach, achieving a mean PSNR of 31.38 dB compared to the supervised baseline of 31.40 dB. As seen in Section 6.1 this small gap in PSNR (only 0.024dB) is in the vast majority of cases, not discernable to the human eye. This shows that the SURE loss provides a valid alternative to the classic self-supervised loss without having to sacrifice the quality of our reconstructions. The R2R and N2N losses, whilst R2R is a theoretically unbiased estimator and N2N is approximately unbiased, tend to overestimate our optimal regularisation parameter α , resulting in more aggressive regularisation and lower reconstruction quality. This consequence is best seen in Figure 10 where a lot of the finer details within the image have been lost due to the aggressive regularisation. This suggests that the R2R and N2N losses may not offer a viable alternative to our supervised baseline due to the loss of finer image details in our reconstructions, which is not ideal especially in real world imaging scenarios where a high quality reconstruction is the main priority. We also explored the case of spatially varying regularisation, where the supervised approach obtained a meaningful improvement of 0.40 dB over the global supervised baseline. Extending self-supervised methods to this setting however was more challenging. The spatially varying SURE approach underperformed due to the increased variance of the SURE estimate at the block level, suggesting that the self-supervised methods for spatially varying approaches still need more investigation.

7.2 Future Work and Possible Extensions

The first possible extensions to this work would be to tune some of the hyper-parameters associated with our self-supervised losses. As seen in Sections 3.3 and 3.4 both our R2R and N2N losses had additional parameters, β for R2R and γ for N2N, that we fixed for the purposes of our project. It would be interesting to also try and optimise these parameters to see what outcome this has on the effectiveness of the R2R and N2N losses, especially to see if this may alleviate some of the over-regularisation issues. Additionally I think that performing some sensitivity analysis on our parameter K for the monte-carlo estimates within the practical implementations would hold some merit. In our current approach we fixed K to be seven as it provided a compromise between effectiveness and computation

time. Increasing K to say 15 or 20 would be a worthwhile extension and would hopefully help to reduce some of the variance we saw in the N2N and R2R losses. It would also be interesting to try and apply the SURE loss to the work in [KAAB⁺23] and [VKP25] to see if its still effective when we use a neural network to create our reconstructions. This extension is especially exciting as in both papers the authors found that their "learning of the parameter map" approach had significant advantages over typical deep learning approaches. It would be nice to see if these advantages, such as the *triple edge structure*, still remain if we switch to a self-supervised loss within the neural network instead.

References

- [Com93] Eastman Kodak Company. Kodak lossless true color image suite. <http://r0k.us/graphics/kodak/>, 1993.
- [CP11] Antonin Chambolle and Thomas Pock. A first-order primal-dual algorithm for convex problems with applications to imaging. *Journal of mathematical imaging and vision*, 40(1):120–145, 2011.
- [HLJ⁺21] Tao Huang, Songjiang Li, Xu Jia, Huchuan Lu, and Jianzhuang Liu. Neighbor2neighbor: Self-supervised denoising from single noisy images. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 14781–14790, 2021.
- [Hut89] Michael F Hutchinson. A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics-Simulation and Computation*, 18(3):1059–1076, 1989.
- [KAAB⁺23] Andreas Kofler, Fabian Altekruiger, Fatima Antarou Ba, Christoph Kolbitsch, Evangelos Papoutsellis, David Schote, Clemens Sirotenko, Felix Frederik Zimmermann, and Kostas Papafitsoros. Learning regularization parameter-maps for variational image reconstruction using deep neural networks and algorithm unrolling. *SIAM Journal on Imaging Sciences*, 16(4):2202–2246, 2023.
- [LMH⁺18] Jaakko Lehtinen, Jacob Munkberg, Jon Hasselgren, Samuli Laine, Tero Karras, Miika Aittala, and Timo Aila. Noise2noise: Learning image restoration without clean data. *arXiv preprint arXiv:1803.04189*, 2018.
- [Pap15] Konstantinos Papafitsoros. *Novel higher order regularisation methods for image reconstruction*. PhD thesis, University of Cambridge, 2015.
- [PZQJ21] Tongyao Pang, Huan Zheng, Yuhui Quan, and Hui Ji. Recorrupted-to-recorrupted: Unsupervised deep learning for image denoising. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 2043–2052, 2021.
- [RBU08] Sathish Ramani, Thierry Blu, and Michael Unser. Monte-Carlo SURE: A black-box optimization of regularization parameters for general denoising algorithms. *IEEE Transactions on Image Processing*, 17(9):1540–1554, 2008.

- [ROF92] Leonid I Rudin, Stanley Osher, and Emad Fatemi. Nonlinear total variation based noise removal algorithms. *Physica D: nonlinear phenomena*, 60(1-4):259–268, 1992.
- [Ste81] Charles M Stein. Estimation of the mean of a multivariate normal distribution. *The annals of Statistics*, pages 1135–1151, 1981.
- [TD26] Julián Tachella and Mike Davies. Self-supervised learning from noisy and incomplete data. *arXiv preprint arXiv:2601.03244*, 2026.
- [Tik63] Andrei N Tikhonov. Solution of incorrectly formulated problems and the regularization method. *Sov Dok*, 4:1035–1038, 1963.
- [VKP25] Thanh Trung Vu, Andreas Kofler, and Kostas Papafitsoros. Deep unrolling for learning optimal spatially varying regularisation parameters for total generalised variation. In *International Conference on Scale Space and Variational Methods in Computer Vision*, pages 282–294. Springer, 2025.
- [WBSS04] Zhou Wang, A.C. Bovik, H.R. Sheikh, and E.P. Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.
- [Whi22] Edmund T Whittaker. On a new method of graduation. *Proceedings of the Edinburgh Mathematical Society*, 41:63–75, 1922.

A Full Results Table

Image	Supervised		SURE		R2R		N2N	
	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM
kodim01	29.47	0.859	29.47	0.859	28.37	0.813	27.56	0.772
kodim02	33.74	0.849	33.73	0.850	32.93	0.831	33.74	0.849
kodim03	33.68	0.893	33.68	0.893	32.74	0.884	33.57	0.896
kodim04	32.39	0.850	32.38	0.851	31.81	0.837	32.26	0.849
kodim05	28.92	0.895	28.88	0.894	28.09	0.871	26.80	0.823
kodim06	30.49	0.823	30.49	0.824	29.37	0.779	29.29	0.774
kodim07	31.06	0.892	31.06	0.894	30.44	0.896	30.64	0.899
kodim08	28.78	0.901	28.77	0.903	27.87	0.892	26.35	0.853
kodim09	32.56	0.891	32.48	0.877	31.77	0.894	32.49	0.899
kodim10	32.18	0.873	32.17	0.871	31.43	0.863	31.84	0.873
kodim11	31.43	0.851	31.42	0.849	30.81	0.838	31.09	0.847
kodim12	33.36	0.862	33.36	0.863	32.76	0.853	32.73	0.852
kodim13	28.87	0.858	28.83	0.856	28.08	0.821	27.23	0.778
kodim14	30.18	0.859	30.17	0.859	29.33	0.826	29.68	0.841
kodim15	33.13	0.892	33.12	0.890	31.97	0.881	32.72	0.892
kodim16	32.31	0.826	32.29	0.825	31.40	0.786	31.96	0.811
kodim17	31.69	0.884	31.68	0.882	31.03	0.872	31.65	0.885
kodim18	30.11	0.845	29.93	0.832	28.97	0.806	28.48	0.783
kodim19	30.94	0.841	30.94	0.842	30.04	0.824	29.49	0.806
kodim20	32.40	0.923	32.33	0.925	31.48	0.912	30.64	0.896
kodim21	30.96	0.876	30.95	0.868	30.00	0.883	30.41	0.891
kodim22	31.54	0.832	31.54	0.831	30.63	0.795	31.08	0.815
kodim23	33.32	0.897	33.30	0.899	32.39	0.892	32.87	0.899
kodim24	30.07	0.861	30.04	0.863	29.00	0.830	29.12	0.835

Table 2: Full PSNR (dB) and SSIM results for all 24 Kodak images across all four methods.